# MODUL PRAKTIKUM 3

## DOCKER CONTAINERIZATION

Pemrograman Sisi Server — A11.64707

Suprayogi,M.Kom

| Informasi | Keterangan |
|---|---|
| Pertemuan | Minggu 3 |
| Sub-CPMK | Menerapkan containerization menggunakan Docker untuk deployment aplikasi |
| Metode | Demo Dosen + Guided Lab + Experiment + Mini Challenge |
| Kasus latihan | Python Web App sederhana + PostgreSQL |
| Capstone | - |

## 1. Posisi Praktikum dalam RPS

Minggu 3 berfokus pada Docker Containerization. Mahasiswa mempelajari Docker fundamentals, Dockerfile best practices, Docker Compose, networking, volumes, serta multi-stage build. Target praktiknya adalah melakukan containerization aplikasi backend dan database dengan environment variables, persistent volume, networking, dan dokumentasi deployment.

**Alur pembelajaran**
Mahasiswa mulai dari container yang sangat sederhana, kemudian membangun image sendiri, memahami volume dan network, menggabungkan aplikasi dengan PostgreSQL melalui Docker Compose, lalu menerapkannya pada Capstone Simple LMS.

## 2. Capaian Praktikum

- Menjelaskan perbedaan image, container, Dockerfile, registry, volume, dan network.
- Menjalankan dan mengelola container menggunakan Docker CLI.
- Membangun custom image dari Dockerfile.
- Menggunakan environment variable tanpa menanam credential di source code.
- Menggunakan volume untuk persistent data.
- Menghubungkan application container dan database container melalui Docker network.
- Menggunakan Docker Compose untuk menjalankan multi-container application.
- Menerapkan prinsip dasar Dockerfile yang reproducible dan efisien.
- Menganalisis fungsi multi-stage build secara konseptual.
- Melakukan containerization environment Capstone Simple LMS.

## 3. Alokasi Aktivitas

| Tahap | Aktivitas | Estimasi |
|---|---|---|
| Konsep | Image, container, Dockerfile, volume, network, Compose | 25 menit |
| Demo dosen | docker run → build image Python sederhana | 25 menit |
| Guided Lab A | Docker CLI & lifecycle | 20 menit |
| Guided Lab B | Dockerfile dan custom image | 30 menit |
| Guided Lab C | Volume & environment variable | 20 menit |
| Guided Lab D | Docker Compose: app + PostgreSQL | 40 menit |
| Mini Challenge | Perbaikan containerization | 20 menit |
| Capstone | Milestone 3 Simple LMS | 10 menit |

## 4. Prasyarat

| Komponen | Kondisi | Verifikasi |
|---|---|---|
| Docker Desktop | Terinstal dan running | docker --version |
| Docker Compose | Tersedia | docker compose version |
| Python | 3.9+ | python --version |
| Git | Terinstal | git --version |
| IDE | VS Code/PyCharm | Buka folder praktikum |
| Port lokal | 8000 dan 5432 idealnya bebas | Periksa jika terjadi konflik |

**Sebelum mulai**
Pastikan Docker Desktop sudah berjalan. Praktikum ini tidak membutuhkan instalasi PostgreSQL langsung pada host karena database akan dijalankan sebagai container.

# 5. Konsep Dasar Docker

## 5.1 Masalah yang Diselesaikan Container

Aplikasi sering berjalan di komputer developer tetapi gagal di komputer lain karena perbedaan versi runtime, dependency, konfigurasi, atau environment. Container membantu mengemas aplikasi beserta kebutuhan runtime-nya dalam environment yang konsisten.

```
Source Code + Dependencies + Runtime
               |
               v
         Docker Image
               |
         docker run
               v
        Docker Container
```

## 5.2 Image vs Container

| Image | Container |
|---|---|
| Blueprint/template read-only | Instance dari image yang sedang/ pernah dijalankan |
| Dibangun dari Dockerfile | Memiliki lifecycle start/stop/remove |
| Dapat diberi tag | Memiliki nama/ID |
| Dapat digunakan membuat banyak container | Perubahan filesystem container tidak ideal untuk data persisten |

## 5.3 Komponen Utama

| Komponen | Fungsi |
|---|---|
| Dockerfile | Instruksi membangun image |
| Image | Artefak aplikasi yang dapat dijalankan |
| Container | Instance runtime dari image |
| Registry | Tempat menyimpan/distribusi image |
| Volume | Persistent storage di luar writable layer container |
| Network | Komunikasi antar-container |
| Docker Compose | Mendefinisikan dan menjalankan multi-container application |

# 6. DEMO DOSEN — Container Pertama

```
d:\mkdir minggu-3

d:\cd minggu-3
```

## 6.1 Verifikasi Docker

```
docker --version
docker compose version
docker info
```

## 6.2 Jalankan hello-world

```
docker run hello-world
```

Docker mencari image lokal, mengambil image bila belum tersedia, membuat container, menjalankan proses, lalu proses selesai.

## 6.3 Lihat Container dan Image

```
docker ps
docker ps -a
docker images
```

**Checkpoint 1**

Mahasiswa harus dapat menjelaskan mengapa `docker ps` dan `docker ps -a` dapat menampilkan hasil yang berbeda.

## 7. GUIDED LAB A — Container Lifecycle

Jalankan web server Nginx sebagai latihan lifecycle container.

```
docker run --name nginx-minggu3 -d -p 8080:80 nginx
docker ps
```

Buka pada browser:

```
http://localhost:8080
```

Kemudian coba:

```
docker stop nginx-minggu3
docker ps
docker ps -a
docker start nginx-minggu3
docker logs nginx-minggu3
docker rm -f nginx-minggu3
```

| Opsi/Perintah | Makna |
|---|---|
| -d | Detached/background mode |
| --name | Memberi nama container |
| -p 8080:80 | Memetakan host port 8080 ke container port 80 |
| docker stop/start | Mengubah state container |
| docker logs | Melihat stdout/stderr container |
| docker rm | Menghapus container |

## 8. GUIDED LAB B — Membangun Custom Python Image

### 8.1 Struktur Project

```
minggu-3/
├── app.py
├── requirements.txt
├── Dockerfile
└── .dockerignore
```

### 8.2 Buat `app.py`

```
from http.server import BaseHTTPRequestHandler, HTTPServer
import json
import os

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/health":
            data = {
                "status": "ok",
                "environment": os.getenv("APP_ENV", "development")
            }
            self.send_response(200)
        else:
            data = {"error": "not found"}
            self.send_response(404)
```

```
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps(data).encode())

HTTPServer(("0.0.0.0", 8000), Handler).serve_forever()
```

Perhatikan penggunaan `0.0.0.0`, bukan `localhost`, agar server dapat menerima koneksi melalui network interface container.

### 8.3 requirements.txt

```
# Aplikasi latihan ini hanya menggunakan Python standard library.
# File tetap dibuat untuk menunjukkan pola dependency project.
```

### 8.4 .dockerignore

```
.git
.venv
__pycache__
*.pyc
.env
```

### 8.5 Dockerfile

```
FROM python:3.12-slim

WORKDIR /app

COPY . .

EXPOSE 8000

CMD ["python", "app.py"]
```

### 8.6 Build dan Run

```
docker build -t app-minggu3:1.0 .
docker run --name app-minggu3 -d -p 8000:8000 app-minggu3:1.0
docker logs  app-minggu3
```

Uji:

```
http://localhost:8000/health
```

**Checkpoint 2**
Mahasiswa harus dapat menjelaskan fungsi `FROM`, `WORKDIR`, `COPY`, `EXPOSE`, dan `CMD` tanpa membaca definisi secara hafalan.

## 9. Dockerfile Best Practices

- Gunakan base image yang sesuai kebutuhan dan versi/tag yang disengaja.
- Susun layer agar dependency yang jarang berubah dapat memanfaatkan build cache.
- Jangan memasukkan `.env`, secret, `.git`, virtual environment, atau file tidak relevan ke build context.
- Gunakan `.dockerignore`.
- Hindari menyimpan data persisten pada writable layer container.
- Gunakan `--no-cache-dir` saat instalasi pip bila sesuai.

- Untuk aplikasi production, pertimbangkan menjalankan proses sebagai non-root user.
- Bedakan development configuration dan production configuration.

**Catatan pembelajaran**
Pada Capstone minggu 3 mahasiswa tidak cukup hanya membuat Dockerfile yang 'bisa jalan'. Mereka harus mampu menjelaskan konsekuensi setiap instruction.

## 10. GUIDED LAB C — Environment Variables

Hentikan container sebelumnya lalu jalankan dengan environment variable:

```
docker rm -f app-minggu3
docker run --name app-minggu3 -d   -p 8000:8000   -e APP_ENV=container-lab    app-minggu3:1.0
```

Uji kembali `/health`. Nilai `environment` harus berubah.

### 10.1 Mengapa Environment Variable?

- Konfigurasi dapat berbeda antar-environment tanpa mengubah source code.
- Memisahkan configuration dari application code.
- Credential dapat diinjeksi saat runtime.
- File `.env` nyata tidak boleh di-commit.

**Security warning**
Environment variable bukan alasan untuk mempublikasikan secret. `.env.example` hanya berisi nama variable dan contoh non-rahasia.

## 11. GUIDED LAB D — Persistent Volume

Sebelum database, pahami konsep volume dengan container sederhana.

```
docker volume create week3-data
docker volume ls
docker volume inspect week3-data
```

Volume akan digunakan oleh PostgreSQL agar data tetap tersedia walaupun database container dibuat ulang. Tanpa persistent volume, lifecycle container dapat menyebabkan data aplikasi hilang.

## 12. GUIDED LAB E — Docker Compose: App + PostgreSQL

### 12.1 Arsitektur Latihan

```
Browser / Host
     |
 localhost:8000
     |
+------------------+
| app container    |
| Python backend   |
+------------------+
        |
        | hostname: db
```

```
        v
+------------------+
| db container     |
| PostgreSQL       |
+------------------+
        |
        v
   named volume
```

## 12.2 Tambahkan Dependency PostgreSQL

Ubah `requirements.txt`:

```
psycopg[binary]
```

Untuk latihan koneksi database, ganti `app.py` dengan:

```
from http.server import BaseHTTPRequestHandler, HTTPServer
import json
import os
import psycopg

def database_status():
    try:
        with psycopg.connect(
            host=os.getenv("DB_HOST", "db"),
            port=os.getenv("DB_PORT", "5432"),
            dbname=os.getenv("DB_NAME", "db-latihan"),
            user=os.getenv("DB_USER", "user-1"),
            password=os.getenv("DB_PASSWORD", "123"),
        ) as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT 1")
                return cur.fetchone()[0] == 1
    except Exception:
        return False

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/health":
            ok = database_status()
            data = {"app": "ok", "database": "ok" if ok else "error"}
            self.send_response(200 if ok else 503)
        else:
            data = {"error": "not found"}
            self.send_response(404)

        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps(data).encode())

HTTPServer(("0.0.0.0", 8000), Handler).serve_forever()
```

## 12.3 Perbaiki Dockerfile untuk Dependency

```
FROM python:3.12-slim

WORKDIR /app
```

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["python", "app.py"]
```

### 12.4 Buat .env

```
DB_NAME=db-latihan
DB_USER=user-1
DB_PASSWORD=123
DB_HOST=db
DB_PORT=5432
```

**Penting**
File `.env` hanya untuk lokal dan harus ada di `.gitignore`. Buat `.env.example` tanpa credential nyata untuk repository.

### 12.5 Buat compose.yaml

```
services:
  app:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      - db

  db:
    image: postgres:17
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

### 12.6 Jalankan Stack

```
docker compose up --build -d
docker compose ps
docker compose logs app
docker compose logs db
```

Uji:

```
http://localhost:8000/health
```

Expected response ketika database siap:

```
{"app": "ok", "database": "ok"}
```

**Catatan depends_on**
`depends_on` mengatur urutan startup dasar, tetapi tidak selalu berarti database sudah siap menerima koneksi. Pada sistem yang lebih matang, gunakan healthcheck/retry strategy.

## 13. Eksperimen — Networking Antar-Container

Docker Compose membuat network default. Service dapat berkomunikasi menggunakan nama service sebagai hostname. Karena itu aplikasi menggunakan `DB_HOST=db`, bukan `localhost`.

```
docker compose exec app python -c "import socket; print(socket.gethostbyname('db'))"
```

**Pertanyaan penting**
Mengapa `localhost` dari dalam app container tidak menunjuk ke PostgreSQL container? Karena `localhost` selalu menunjuk ke container itu sendiri.

## 14. Eksperimen — Membuktikan Persistent Data

1. Pastikan stack berjalan.
2. Masuk ke PostgreSQL dan buat data uji.
3. Hentikan serta hapus container dengan `docker compose down`.
4. Jalankan kembali `docker compose up -d`.
5. Periksa apakah data masih ada karena named volume tetap dipertahankan.

```
docker compose exec db psql -U user-1 -d db-latihan
CREATE TABLE demo (id SERIAL PRIMARY KEY, message TEXT);
INSERT INTO demo(message) VALUES ('persistent data');
SELECT * FROM demo;
\q

docker compose down
docker compose up -d

docker compose exec db psql -U user-1 -d db-latihan -c "SELECT * FROM demo;"
```

**Hati-hati**
`docker compose down -v` juga menghapus named volume. Jangan gunakan opsi `-v` jika ingin mempertahankan data.

## 15. Multi-Stage Build — Konsep

Jika aplikasi Python kita masih sederhana dan tidak butuh instalasi yang rumit, manfaat teknik **multi-stage builds** ini mungkin belum terlalu terasa. Namun, konsep dasarnya sangat penting: kita membagi proses menjadi dua tempat. Tempat pertama khusus untuk mengunduh dan merakit semua bahan (dependensi), sedangkan tempat kedua (hasil akhir) hanya diisi oleh aplikasi matang yang siap dijalankan tanpa membawa 'sampah' atau alat rakitnya lagi

```
FROM ... AS builder
# build/install artifacts

FROM ... AS runtime
# copy only required runtime artifacts
# run application
```

## 16. Ringkasan

Mahasiswa harus memahami bahwa Docker bukan sekadar cara menjalankan perintah `docker compose up`. mhs harus memahami bagaimana image dibangun, bagaimana container berkomunikasi, bagaimana konfigurasi diinjeksi, mengapa database memerlukan persistent storage, dan bagaimana seluruh environtment dapat direproduksi. Environment ini menjadi fondasi Minggu 4, ketika core Simple LMS mulai diimplementasikan menggunakan Django ORM.